Azure DevOps

Complete 20-Day Training Notes

Beginner to Job-Ready

Covers: DevOps Culture | Azure Boards | Azure Repos | CI/CD Pipelines
YAML | Agents | Artifacts | Docker | Kubernetes | Terraform | AZ-400 Prep

Free account needed: dev.azure.com

Day 1: What is DevOps & Azure DevOps? — Foundation

1. The problem before DevOps

Software teams used to be split into Dev (developers who write code) and Ops (operations who manage servers). Devs wanted to release fast; Ops wanted stability. This conflict caused slow releases, outages, and blame culture.

2. What is DevOps?

DevOps is a culture + set of practices that unites Development and Operations to deliver software faster, more reliably, and with higher quality.
Three words summarize it: Collaborate. Automate. Deliver.

NOTE: DevOps is NOT just a tool. It is a mindset first, tools second.

3. The DevOps lifecycle (8 stages)

Plan	Define features, user stories, sprints using boards
Code	Write code, commit to version control (Git)
Build	Compile code, run automated builds (CI)
Test	Run automated unit, integration, UI tests
Release	Package the build artifact for deployment
Deploy	Push to dev / staging / production environments (CD)
Operate	Monitor running application in production
Monitor	Collect metrics, logs, alerts — feed back into planning

4. What is Azure DevOps?

Azure DevOps is Microsoft's cloud platform that provides all the tools you need to practice DevOps in one place. It is free for small teams and available at dev.azure.com.

5. The 5 services of Azure DevOps

Azure Boards	Plan work: tasks, bugs, sprints, Kanban boards
Azure Repos	Store and manage code using Git
Azure Pipelines	Build and deploy automatically (CI/CD)
Azure Test Plans	Manual and automated testing management
Azure Artifacts	Store and share code packages (NuGet, npm, etc.)

6. Key terms to know

CI – Continuous Integration	Automatically build and test code every time a developer pushes
CD – Continuous Delivery	Automatically deliver the tested build to an environment
Pipeline	A series of automated steps to build/test/deploy code
Repository (Repo)	A folder where all your code is stored with version history
Sprint	A fixed time period (usually 2 weeks) to complete planned work
Build Agent	A machine that runs your pipeline steps
Artifact	The output of a build (e.g. a .zip, .jar, or Docker image)
Environment	A deployment target: Dev, Staging, or Production

7. DevOps vs Traditional (Waterfall)

Waterfall: release cycle	6–12 months
DevOps: release cycle	Hours to days
Waterfall: teams	Siloed Dev and Ops
DevOps: teams	Unified, collaborative
Waterfall: testing	At the end, manual
DevOps: testing	Continuous, automated
Waterfall: failure risk	Very high (big bang releases)
DevOps: failure risk	Low (small frequent releases)

8. Practice task for Day 1

- Go to dev.azure.com and create a free account
- Create a new Organization (e.g. 'MyDevOpsTraining')
- Create a new Project inside it (e.g. 'Day1Practice')
- Explore the left sidebar — you will see Boards, Repos, Pipelines, Test Plans, Artifacts

Today's cheat sheet	
DevOps	Culture of collaboration between Dev and Ops
CI	Auto build + test on every code push
CD	Auto deploy tested code to environment
Azure DevOps URL	dev.azure.com
5 services	Boards, Repos, Pipelines, Test Plans, Artifacts
Sprint	Fixed time box (2 weeks) to deliver work

Day 2: Azure DevOps Overview — Organization, Projects & Navigation

1. Azure DevOps structure

Azure DevOps is organized in a hierarchy: Organization → Project → Services (Boards, Repos, Pipelines...).

Organization	Your company or team's top-level account. Has a unique URL: dev.azure.com/YourOrg
Project	A container for one application or product. Has its own repos, boards, pipelines.
Team	A group of people inside a project with their own board and settings.
Member	A user added to the organization with a specific access level.

2. Access levels

Stakeholder (Free)	Can view boards, add work items, no code/pipeline access
Basic (Free for 5 users)	Full access to Boards, Repos, Pipelines
Basic + Test Plans (Paid)	Adds Azure Test Plans access
Visual Studio subscriber	Full access, included with VS subscription

3. Creating an organization

- Go to dev.azure.com and sign in with a Microsoft account
- Click 'New organization' and choose a name (e.g. MyTrainingOrg)
- Select a region (choose one near you — e.g. South Asia for India)
- Your org URL will be: dev.azure.com/MyTrainingOrg

4. Creating a project

- Click '+ New project' inside your organization
- Give it a name (e.g. WebApp)
- Set visibility: Private (only invited members) or Public (anyone can see)
- Choose version control: Git (always choose Git for modern projects)
- Choose work item process: Agile, Scrum, or CMMI — choose Agile for now

5. Process types explained

Agile	Uses Epics, Features, User Stories, Tasks, Bugs. Best for most teams.
Scrum	Uses Epics, Features, Product Backlog Items, Tasks, Bugs. Sprint-focused.

CMMI	More formal, adds Change Requests and Reviews. Used in regulated industries.
Basic	Simplest: just Issues and Tasks. Good for learning.

6. Navigation overview

Left sidebar icons	Quick access to Boards, Repos, Pipelines, Test Plans, Artifacts
Project Settings (bottom left)	Configure teams, permissions, service connections, billing
Organization Settings	Manage users, billing, policies across all projects
Search bar (top)	Search code, work items, or wiki across the project

7. Practice task for Day 2

- Create 3 projects in your org: WebApp, MobileApp, InfraProject
- In WebApp, go to Project Settings > Teams — notice the default team
- Invite another email as a Stakeholder and see what they can access
- Explore the Organization Settings page

Today's cheat sheet	
Org URL format	dev.azure.com/YourOrgName
Default process	Agile (recommended for beginners)
Private project	Only invited users can see it
Public project	Anyone on the internet can view (read-only)
Max free users (Basic)	5 users per organization
Project Settings	Bottom-left gear icon in the sidebar

Day 3: Azure Boards — Work Items, Sprints & Kanban

1. What is Azure Boards?

Azure Boards is a project management tool inside Azure DevOps. It helps teams plan, track, and discuss work throughout the development lifecycle — similar to Jira or Trello.

2. Work item types (Agile process)

Epic	A very large piece of work spanning multiple sprints (e.g. 'Build User Authentication')
Feature	A chunk of an Epic (e.g. 'Login with Google')
User Story	A specific user need: 'As a user, I want to reset my password so that I can regain access'
Task	A technical step to complete a User Story (e.g. 'Create password reset API endpoint')
Bug	A defect found during testing or production
Test Case	A scenario to validate a feature works correctly

3. Work item states (Agile)

New	Just created, not yet started
Active	Currently being worked on
Resolved	Developer says it is done
Closed	Verified and accepted
Removed	Will not be worked on

4. The Kanban board

The Kanban board shows work items as cards in columns. Each column represents a state. You drag cards left to right as work progresses. It gives instant visual status of your entire team's work.

- Go to Boards > Boards to see the Kanban view
- Columns default to: New, Active, Resolved, Closed
- You can add custom columns in Board Settings

5. Sprints and backlogs

Product Backlog	The full list of all work items not yet scheduled
Sprint Backlog	Work items committed for the current sprint

Sprint	A time-boxed period (1–4 weeks) to complete selected work
Velocity	How many story points a team completes per sprint on average
Story Points	Relative effort estimate for a User Story (1, 2, 3, 5, 8, 13...)

6. Creating a sprint

- Go to Boards > Sprints
- Click 'New Sprint' and set a name (e.g. Sprint 1), start date, and end date
- Drag items from the backlog into the sprint
- During the sprint, update work item states as you progress
- Use the burndown chart to track remaining work over time

7. Queries

Queries let you search and filter work items. Go to Boards > Queries. You can ask things like: 'Show me all Active Bugs assigned to me'. Queries can be saved and shared with the team.

8. Practice task for Day 3

- Create an Epic: 'User Management System'
- Under it, create a Feature: 'User Login'
- Create 3 User Stories under that Feature
- Break each User Story into 2–3 Tasks
- Create a Sprint and assign the stories to it
- Move one item through New > Active > Resolved > Closed on the Kanban board

Today's cheat sheet	
Epic > Feature > Story > Task	Work item hierarchy in Agile process
Kanban board	Boards > Boards (drag cards by state)
Sprint view	Boards > Sprints
Backlog	Boards > Backlogs
Story point scale	1, 2, 3, 5, 8, 13 (Fibonacci-like)
Burndown chart	Shows remaining work vs time in a sprint

Day 4: Azure Repos & Git — Version Control, Branches & Pull Requests

1. What is version control?

Version control tracks every change ever made to your code. Think of it as a 'save history' for code — you can go back to any point, see who changed what, and merge changes from multiple people without conflicts.

2. What is Git?

Git is the most popular version control system in the world. It is distributed — every developer has a full copy of the code history on their machine. Azure Repos hosts Git repositories in the cloud.

3. Key Git concepts

Repository (Repo)	The folder that contains all your code + its full history
Commit	A saved snapshot of your code changes with a message describing what changed
Branch	A parallel version of your code where you can work without affecting others
Merge	Combining changes from one branch into another
Pull Request (PR)	A request to review your branch changes before merging into main
Clone	Downloading a repo from the cloud to your local machine
Push	Uploading your local commits to the cloud repo
Pull	Downloading latest changes from the cloud to your local machine
main / master	The primary branch — always contains stable, working code

4. Essential Git commands

Setup (run once):

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```

Daily workflow:

```
git clone <repo-url>          # Download repo to local machine
git status                    # See what files changed
git add .                     # Stage all changes
git commit -m "Your message" # Save snapshot with message
git push                      # Upload to cloud
git pull                      # Get latest from cloud
```

Branching:

```
git branch feature/login     # Create a new branch
git checkout feature/login   # Switch to that branch
git checkout -b feature/login # Create AND switch in one command
git merge feature/login      # Merge branch into current branch
```

5. Branching strategy (Git Flow)

main	Always deployable, stable code. Protected — no direct pushes.
develop	Integration branch where features are merged for testing.
feature/xxx	Short-lived branch for a specific feature or task.
hotfix/xxx	Emergency fix branched directly from main.
release/xxx	Prep branch before releasing a version to production.

6. Pull Requests in Azure Repos

A Pull Request (PR) is how code gets reviewed before being merged. It is not a 'pull from the internet' — it is a request asking teammates to review your code.

- Developer creates a branch and writes code
- Developer pushes the branch and opens a PR in Azure Repos
- Reviewer is assigned — they leave comments, approve, or request changes
- Once approved, the PR is merged into main
- The feature branch is then deleted

7. Branch policies in Azure Repos

Branch policies protect important branches (like main). You can require: minimum number of reviewers, all comments resolved, work item linked, and successful build before merge.

8. Practice task for Day 4

- In Azure Repos, clone your repo to your local machine using VS Code
- Create a branch called feature/add-homepage
- Add a file called index.html with some content
- Commit and push the branch
- Open a Pull Request in Azure DevOps and review it yourself
- Complete (merge) the PR

Today's cheat sheet	
git clone <url>	Copy cloud repo to local machine
git add . && git commit -m	Stage and save changes
git push / git pull	Upload / download from cloud
feature/xxx branch	Where you do all new work
main branch	Protected — only merge via PR

Pull Request	Code review step before merging

Day 5: Azure Artifacts — Package Feeds, NuGet & npm

1. What is Azure Artifacts?

Azure Artifacts is a package management service. It stores libraries and packages your code depends on — like npm packages for JavaScript or NuGet packages for .NET — in a private, secure feed hosted by Azure.

2. What is a package?

A package is reusable code bundled and published so other projects can depend on it. Instead of copying code across projects, you publish it as a package (e.g. v1.0.0) and reference it as a dependency.

NuGet	Package format for .NET / C# projects (.nupkg files)
npm	Package format for JavaScript / Node.js projects
Maven	Package format for Java projects
PyPI	Package format for Python projects
Universal Package	Any file type — zips, binaries, etc.

3. What is a Feed?

A Feed is a private container for your packages in Azure Artifacts. Think of it like your own private npm registry or NuGet gallery. You publish packages to the feed and consume them in your projects.

4. Upstream sources

A Feed can be configured to proxy public package sources like npmjs.com or nuget.org. This means your team always fetches packages through your Azure Artifacts feed — providing caching, security scanning, and control.

5. Creating a feed (steps)

- Go to Artifacts in your Azure DevOps project
- Click '+ Create Feed'
- Name it (e.g. MyCompanyFeed)
- Choose visibility: project-scoped or organization-scoped
- Add upstream sources (public npm or NuGet) if needed
- Click Create

6. Publishing and consuming packages

Publish a NuGet package:

```
dotnet pack                                  # Create the .nupkg file
dotnet nuget push *.nupkg --source MyFeed    # Push to Azure Artifacts
```

Consume in a project (restore):

```
dotnet restore                          # Downloads packages from feed
```

Publish an npm package:

```
npm publish --registry
https://pkgs.dev.azure.com/YourOrg/_packaging/MyFeed/npm/registry/
```

7. Versioning packages (Semantic Versioning)

MAJOR (1.0.0 → 2.0.0)	Breaking change — existing code may break
MINOR (1.0.0 → 1.1.0)	New feature added, backward compatible
PATCH (1.0.0 → 1.0.1)	Bug fix only, fully backward compatible

NOTE: Always version your packages. Never overwrite an existing version — publish a new one.

8. Practice task for Day 5

- Create a feed called TrainingFeed in your Azure DevOps project
- Add nuget.org as an upstream source
- Browse available packages in the feed
- Read the Connect to Feed instructions for NuGet or npm

Today's cheat sheet	
Azure Artifacts	Private package storage for NuGet, npm, Maven, PyPI
Feed	Your private package registry
Upstream source	Proxy to public registries (npmjs.com, nuget.org)
Semantic version	MAJOR.MINOR.PATCH (e.g. 2.1.3)
dotnet pack	Creates a NuGet package
npm publish	Publishes npm package to feed

Day 6: Azure Test Plans — Manual Testing & Test Cases

1. What is Azure Test Plans?

Azure Test Plans is a testing management tool. It lets you create test cases, organize them into test suites, execute tests manually, and track results — all linked to your work items and bugs.

2. Testing terminology

Test Plan	A container for all testing activity in a sprint or release
Test Suite	A group of test cases (e.g. 'Login Tests', 'Checkout Tests')
Test Case	A specific scenario with steps to verify a feature works
Test Step	One action + expected result (e.g. Click Login > Page redirects to dashboard)
Test Run	An execution session where you actually run test cases and record results
Test Result	Outcome of a test case: Passed, Failed, or Blocked
Bug	A work item created when a test case fails

3. Creating a test plan

- Go to Test Plans in your project
- Click '+ New Test Plan'
- Give it a name (e.g. Sprint 1 Test Plan) and set iteration/sprint
- A default test suite is created automatically
- Add more suites to organize by feature area

4. Creating test cases

- Inside a test suite, click '+ New Test Case'
- Add a title (e.g. 'Verify user can log in with valid credentials')
- Add test steps: each step has an Action and Expected Result
- Example step — Action: Enter username admin@test.com, Expected Result: No error shown
- Link the test case to a User Story work item for traceability

5. Running tests manually

- Select test cases in the suite, click 'Run' > 'Run for web application'
- A test runner panel opens alongside the app
- Go through each step, mark it Passed or Failed
- If failed, click 'Create Bug' — it auto-fills bug with screenshots and steps
- Save the test run results

6. Test metrics and reporting

Pass rate	% of test cases that passed in this run
Test coverage	% of User Stories/requirements covered by test cases
Bug count by severity	How many Critical, High, Medium, Low bugs found
Traceability matrix	Which stories have which test cases — gaps are visible

7. Practice task for Day 6

- Create a Test Plan called Sprint 1 Plan
- Create 2 Test Suites: Login Tests and Registration Tests
- Write 3 test cases for Login Tests with at least 3 steps each
- Run the test cases and mark some as Passed, some as Failed
- Create a Bug from a failed test case

Today's cheat sheet	
Test Plan	Container for a sprint/release's testing activity
Test Suite	Group of related test cases
Test Case	Step-by-step scenario to verify one feature
Test Run	Actual execution of test cases
Pass / Fail / Blocked	Three possible test results
Bug from test	Click 'Create Bug' when a test step fails

Day 7: YAML Basics — Syntax for Azure Pipelines

1. What is YAML?

YAML stands for Yet Another Markup Language (or YAML Ain't Markup Language). It is a human-readable format for writing configuration files. Azure Pipelines uses YAML to define your entire CI/CD pipeline as code stored in your repository.

2. YAML rules — CRITICAL

- Indentation uses SPACES only — NEVER use tabs. 2 spaces per level is standard.
- YAML is case-sensitive. 'name' and 'Name' are different keys.
- Strings with special characters must be quoted: 'Hello: World'
- Lists use a dash (-) followed by a space
- Key-value pairs use a colon followed by a space: key: value
- Comments start with #

3. Basic YAML syntax examples

Key-value pairs:

```
name: MyPipeline
version: 1
enabled: true
```

Lists:

```
fruits:
  - apple
  - banana
  - mango
```

Nested objects:

```
server:
  host: localhost
  port: 8080
  secure: false
```

Multi-line string:

```
script: |
  echo Hello
  echo World
```

4. Azure Pipeline YAML structure

```
trigger:            # When to run the pipeline
  - main            # Run when main branch is updated

pool:               # What machine to run on
  vmImage: ubuntu-latest

stages:             # Top-level groupings
  - stage: Build
```

```
jobs:           # Work units inside a stage
  - job: BuildJob
    steps:      # Individual tasks/scripts
      - script: echo Hello World
        displayName: Print hello
```

5. YAML hierarchy in pipelines

Pipeline	The whole YAML file — defines the full workflow
Stage	A major phase: Build, Test, Deploy. Stages run sequentially.
Job	A unit of work inside a stage. Jobs can run in parallel.
Step	A single action inside a job: run a script or a task
Task	A pre-built action from Microsoft or the community (e.g. DotNetCoreCLI@2)
Script	A raw command-line command (bash, PowerShell, cmd)

6. Common YAML mistakes

Using tabs instead of spaces	Pipeline fails with 'mapping values are not allowed' error
Wrong indentation level	Steps appear under wrong job or stage
Missing space after colon	key:value is invalid; key: value is correct
String with colon not quoted	message: Hello: World breaks — use quotes

7. Practice task for Day 7

- Open Notepad or VS Code and write a YAML file representing your personal info
- Include: name, age, skills (list), address (nested object)
- Validate it at yamllint.com
- In Azure DevOps, go to Pipelines > New Pipeline and view the starter YAML

Today's cheat sheet	
Indentation	2 spaces (NEVER tabs)
List item	Starts with dash-space (-)
Comment	Starts with #
Multi-line script	Use | (pipe) after the key
Pipeline order	Pipeline > Stage > Job > Step
Trigger	When the pipeline auto-runs (e.g. on push to main)

Day 8: Agents & Pools — Hosted vs Self-Hosted Agents

1. What is an agent?

An agent is a machine (physical or virtual) that runs your pipeline steps. When you define a pipeline and it is triggered, Azure DevOps assigns an agent to execute each job. The agent downloads your code, runs the steps, and reports results back.

2. Microsoft-hosted agents

Microsoft provides free cloud VMs that are pre-configured with popular tools. Every job gets a fresh VM — it is destroyed after the job finishes. You get 1,800 free pipeline minutes per month.

ubuntu-latest	Ubuntu Linux — fast, most common for CI. 7 GB RAM.
windows-latest	Windows Server — needed for .NET Framework, Windows apps.
macos-latest	macOS — needed for iOS/macOS app builds.

Pre-installed tools on ubuntu-latest include:

- Git, Docker, Node.js, Python, Java, .NET, kubectl, Terraform, Azure CLI

3. Self-hosted agents

A self-hosted agent is a machine you control and register with Azure DevOps. You install the agent software on it, and it waits for jobs. Use self-hosted when you need: special software, faster builds (no VM startup time), access to internal networks, or unlimited free minutes.

4. Setting up a self-hosted agent (Linux)

Step 1: Get a Personal Access Token (PAT)

- Go to User Settings > Personal Access Tokens
- Click New Token, set scope to Agent Pools: Read & Manage
- Copy the token — you will not see it again

Step 2: Download and configure the agent

```
mkdir myagent && cd myagent
curl -O https://vstsagentpackage.azureedge.net/agent/3.x.x/vsts-agent-linux-x64-3.x.x.tar.gz
tar zxvf vsts-agent-linux-x64-*.tar.gz
./config.sh
# Enter your org URL: https://dev.azure.com/YourOrg
# Enter your PAT when prompted
# Enter pool name: Default
./run.sh    # Start the agent
```

5. Agent pools

An agent pool is a collection of agents. Pipelines target a pool, not an individual agent. Azure DevOps picks any available agent from that pool to run your job.

Azure Pipelines pool	Microsoft-hosted agents — ubuntu, windows, macos
Default pool	Where self-hosted agents are registered by default
Custom pools	You can create named pools for specific teams or environments

6. Choosing between hosted and self-hosted

Use hosted when	No special requirements, open-source project, quick setup needed
Use self-hosted when	Need corporate network access, special tools, unlimited minutes, faster builds

7. Practice task for Day 8

- Go to Organization Settings > Agent Pools to see the default pools
- Create a simple pipeline using pool: vmImage: ubuntu-latest
- Run the pipeline and watch the job logs to see what the hosted agent does
- Read about self-hosted agent setup in the Azure DevOps docs

Today's cheat sheet	
Microsoft-hosted agent	Azure-managed VM, fresh each run, 1800 free mins/month
Self-hosted agent	Your own machine running the agent software
ubuntu-latest	Most common hosted image for CI pipelines
Agent pool	Group of agents — pipeline targets a pool
PAT	Personal Access Token — used to auth the agent to your org
./config.sh	Command to register a self-hosted agent

Day 9: CI Pipelines — Build, Test & Continuous Integration

1. What is Continuous Integration (CI)?

CI means automatically building and testing your code every time a developer pushes to the repository. The goal: catch bugs early, before they reach production. A CI pipeline runs in minutes and gives developers instant feedback.

2. CI pipeline YAML structure

```yaml
trigger:
  branches:
    include:
      - main
      - develop

pool:
  vmImage: ubuntu-latest

variables:
  buildConfiguration: Release

stages:
  - stage: Build
    displayName: Build and Test
    jobs:
      - job: BuildJob
        steps:
          - task: UseDotNet@2
            inputs:
              packageType: sdk
              version: 8.x

          - script: dotnet restore
            displayName: Restore packages

          - script: dotnet build --configuration $(buildConfiguration)
            displayName: Build application

          - script: dotnet test --no-build
            displayName: Run unit tests

          - task: PublishBuildArtifacts@1
            inputs:
              PathtoPublish: $(Build.ArtifactStagingDirectory)
              ArtifactName: drop
```

3. Trigger types

Push trigger	Run pipeline when code is pushed to specified branches
Pull Request trigger (PR)	Run pipeline when a PR is opened — validates before merge

Scheduled trigger	Run at specific times (e.g. every night at 2am)
Manual trigger	Run only when a human clicks 'Run pipeline'

4. Important built-in variables

$(Build.BuildId)	Unique ID of the current build run
$(Build.BuildNumber)	Human-readable build number (e.g. 20240101.1)
$(Build.SourceBranch)	Branch that triggered the build (e.g. refs/heads/main)
$(Build.Repository.Name)	Name of the Azure Repos repo
$(Build.ArtifactStagingDirectory)	Temp folder for storing build outputs
$(System.DefaultWorkingDirectory)	Root folder where code is checked out
$(Agent.OS)	Operating system of the agent (Windows_NT, Linux, Darwin)

5. Common CI tasks

UseDotNet@2	Install a specific .NET SDK version
NodeTool@0	Install a specific Node.js version
NuGetCommand@2	Restore NuGet packages
DotNetCoreCLI@2	Build, test, publish .NET apps
PublishTestResults@2	Publish test results to Azure DevOps
PublishBuildArtifacts@1	Save build output as a downloadable artifact
Docker@2	Build and push Docker images

6. Build artifacts

A build artifact is the output of your CI pipeline — for example, a compiled binary, a zip file, or a Docker image. Artifacts are stored and can be downloaded or used by the CD pipeline for deployment.

7. Practice task for Day 9

- Create a new pipeline using the starter YAML template
- Change the trigger to run on pushes to main
- Add a step that prints environment information: echo Build ID: $(Build.BuildId)
- Add a step that echoes the branch name
- Run the pipeline and inspect the logs of each step

Today's cheat sheet

CI	Auto build + test on every push
trigger: branches	Which branches auto-trigger the pipeline
pool: vmImage	Specifies the hosted agent image
script:	Raw shell command in a step
task:	Pre-built action (e.g. DotNetCoreCLI@2)
PublishBuildArtifacts@1	Saves compiled output as artifact for CD

Day 10: CD Release Pipelines — Deploy to Environments with Approvals

1. What is Continuous Delivery (CD)?

CD automatically deploys the artifact produced by CI to target environments (Dev, Staging, Production). The goal: every successful build can be deployed with one click — or fully automatically.

2. Two ways to define CD in Azure DevOps

Classic Release Pipelines (GUI)	Visual editor with drag-and-drop stages. Easier for beginners. Separate from the CI YAML.
YAML multi-stage pipelines	CI and CD defined together in one YAML file. Modern approach. More control.

3. YAML multi-stage pipeline (CI + CD together)

```
stages:
  - stage: Build
    jobs:
      - job: BuildApp
        steps:
          - script: echo Building...
          - publish: $(Build.ArtifactStagingDirectory)
            artifact: drop

  - stage: DeployDev
    dependsOn: Build
    condition: succeeded()
    jobs:
      - deployment: DeployToDev
        environment: Development
        strategy:
          runOnce:
            deploy:
              steps:
                - script: echo Deploying to Dev...

  - stage: DeployProd
    dependsOn: DeployDev
    condition: succeeded()
    jobs:
      - deployment: DeployToProduction
        environment: Production
        strategy:
          runOnce:
            deploy:
              steps:
                - script: echo Deploying to Production...
```

4. Deployment strategies

runOnce	Deploy once to all targets. Simple, no rollback logic.
rolling	Replace old instances gradually — reduces downtime.
canary	Deploy to a small % of servers first, then all. Safe for risky releases.
blue-green	Two identical environments — switch traffic when new version is ready.

5. Approvals and gates

Approvals require a human to approve a deployment before it proceeds. Gates are automated checks (e.g. query an API, check a metric) that must pass before deployment continues.

- In Azure DevOps, go to Environments and select an environment
- Click '...' > Approvals and Checks
- Add an Approval and assign approvers
- When the pipeline reaches that stage, it pauses and sends an email to approvers
- Approver clicks Approve or Reject in Azure DevOps

6. Environments

An Environment in Azure DevOps is a logical target for deployment (Dev, Staging, Production). It tracks deployment history, lets you set approvals, and shows which version is currently deployed where.

7. Practice task for Day 10

- Create two Environments in Azure DevOps: Development and Production
- Add yourself as an approval required for the Production environment
- Create a multi-stage YAML pipeline: Build > DeployDev > DeployProd
- Run it and approve the production deployment when prompted

Today's cheat sheet	
CD	Auto-deploy tested artifacts to target environments
Environments	Pipelines > Environments (Dev, Staging, Prod)
dependsOn:	Stage only runs after the named stage succeeds
condition: succeeded()	Only run this stage if previous stage passed
deployment job	Special job type that records to environment history
Approval	Human gate before deploying to a critical environment

Day 11: Variables & Secrets — Pipeline Variables, Groups & Key Vault

1. Types of variables in Azure Pipelines

Inline YAML variables	Defined in the YAML file under variables: section
Pipeline UI variables	Defined in the pipeline settings in Azure DevOps UI
Variable groups	Shared collection of variables, reusable across pipelines
Secret variables	Encrypted at rest, not shown in logs — use for passwords, keys
Azure Key Vault secrets	Secrets stored in Azure Key Vault, linked to variable group

2. Defining variables in YAML

```
variables:
  buildConfig: Release
  appName: MyWebApp
  # Reference with $(variableName)

steps:
  - script: echo Building $(appName) in $(buildConfig) mode
```

3. Variable groups

A variable group is a named set of variables you define once and reuse across multiple pipelines. Great for storing shared config like database connection strings or API URLs.

- Go to Pipelines > Library > + Variable Group
- Name it (e.g. SharedConfig)
- Add variables: AppUrl, DatabaseHost, etc.
- Mark sensitive variables as Secret (lock icon)
- In YAML, reference a variable group:

```
variables:
  - group: SharedConfig
  - name: buildConfig
    value: Release
```

4. Secret variables

Secret variables are encrypted and never shown in pipeline logs. Use them for passwords, API keys, and connection strings.

- In pipeline UI, go to Variables tab and add a variable
- Click the lock icon to make it a secret
- Reference it in YAML as $(MySecret) — it appears as *** in logs

NOTE: Never hard-code secrets in your YAML files! Anyone with repo access can read them.

5. Azure Key Vault integration

Azure Key Vault is a cloud service that stores secrets, keys, and certificates securely. You can link a Key Vault to a Variable Group so pipeline secrets are fetched from Key Vault at runtime.

- Create an Azure Key Vault in the Azure Portal
- Add secrets to it (e.g. DatabasePassword, ApiKey)
- In Azure DevOps, create a Variable Group and toggle 'Link secrets from Azure Key Vault'
- Select your subscription and Key Vault, then choose which secrets to expose
- Reference them in YAML: $(DatabasePassword)

6. Runtime parameters vs variables

Variables	Set values at pipeline definition time. Can be overridden at queue time.
Parameters	Strongly typed inputs defined at the top of YAML — user fills them in when manually running the pipeline.

```
parameters:
  - name: environment
    displayName: Target Environment
    type: string
    default: dev
    values:
      - dev
      - staging
      - production
```

7. Practice task for Day 11

- Create a Variable Group called AppSettings with variables AppName and AppVersion
- Create a pipeline that references this group and prints both variables
- Add a secret variable for a fake password — verify it shows as *** in logs

Today's cheat sheet	
$(variableName)	Reference any variable in YAML or scripts
Variable group	Pipelines > Library > Variable Group
Secret variable	Encrypted — shows as *** in logs
Azure Key Vault link	Key Vault secrets accessible in pipeline via variable group
parameters:	Runtime inputs user provides when triggering pipeline manually
Never commit secrets	Use variable groups or Key Vault instead

Day 12: Templates & Reusability — Pipeline Templates and Extends

1. Why use templates?

Without templates, you copy the same YAML steps across 10 pipelines. When you need to change one step, you update 10 files. Templates solve this: define steps once in a template file, reference it from many pipelines.

2. Types of templates

Step template	A reusable set of steps inserted into a job
Job template	A reusable job (with its own steps) inserted into a stage
Stage template	A reusable stage inserted into a pipeline
Variable template	A file containing shared variable definitions

3. Step template example

Create file: templates/build-steps.yml

```
parameters:
  - name: configuration
    type: string
    default: Release

steps:
  - script: dotnet restore
    displayName: Restore

  - script: dotnet build --configuration ${{ parameters.configuration }}
    displayName: Build

  - script: dotnet test
    displayName: Test
```

Reference from main pipeline YAML:

```
stages:
  - stage: Build
    jobs:
      - job: BuildJob
        steps:
          - template: templates/build-steps.yml
            parameters:
              configuration: Release
```

4. The extends keyword

The extends keyword enforces that all pipelines must use a specific template. This is used by security-conscious organizations to ensure every pipeline meets company standards (e.g. runs a security scan step).

```
extends:
```

```
  template: templates/secure-pipeline.yml
  parameters:
    appName: MyApp
```

5. Variable templates

Create file: templates/vars.yml

```
variables:
  appName: MyWebApp
  environment: production
  timeout: 30
```

Reference in pipeline:

```
variables:
  - template: templates/vars.yml
  - name: buildConfig
    value: Release
```

6. Template expressions vs runtime expressions

${{ expression }}	Compile-time — evaluated when YAML is parsed, before the pipeline runs
$(expression)	Runtime — evaluated when the step actually runs

NOTE: Use ${{ parameters.name }} for template parameters. Use $(variableName) for pipeline variables.

7. Practice task for Day 12

- Create a templates/ folder in your repo
- Create build-steps.yml with a restore, build, and echo step
- Create a main pipeline that uses this template with a parameter
- Run it and verify the template steps execute correctly

Today's cheat sheet	
Template file	A YAML file with reusable steps/jobs/stages
- template: path.yml	Include a template in your pipeline
parameters:	Pass values into a template
${{ parameters.name }}	Reference a template parameter
extends:	Force all pipelines to build from a base template
Variable template	Shared variables file included with - template:

Day 13: Environments & Deployment Jobs — Approvals, History & Strategies

1. Environments in depth

An Azure DevOps Environment is more than a deployment target — it is a record of every deployment that has ever gone to that logical place. You can see: when each version was deployed, who approved it, and whether it succeeded.

2. Creating environments

- Go to Pipelines > Environments > + New Environment
- Give it a name (e.g. Production) and an optional description
- Select resource type: None (for general deployments), Kubernetes, or Virtual Machines
- Environments with Kubernetes or VM resources give you agent-level control

3. Checks and approvals on environments

Approvals	A named person must approve before the stage continues
Branch control	Pipeline must be running from an approved branch (e.g. main)
Business hours	Deployment only allowed during certain hours/days
Invoke Azure Function	Call an API and it must return success before proceeding
Query Work Items	Block deployment if there are open P1 bugs in the board

4. Deployment job YAML

```
jobs:
  - deployment: DeployToProduction
    displayName: Deploy to Production
    environment: Production
    strategy:
      runOnce:
        deploy:
          steps:
            - download: current
              artifact: drop

            - script: echo Deploying version $(Build.BuildNumber)
              displayName: Deploy application
```

5. Deployment strategies explained

runOnce	All instances updated in one go. Simplest. Used for dev/staging.
rolling	Update one VM/instance at a time. Reduces downtime. Used for medium-risk deploys.

canary	Deploy to 10% of instances first. If healthy, deploy to 100%. Safest.

6. VM resource targets

For environments targeting Virtual Machines, install the Azure Pipelines agent on each VM and register it with the environment. The deployment job then runs on those specific VMs directly.

7. Environment history and traceability

Every deployment job that runs against an environment is recorded: build number, commit, branch, date, approver, and result. This gives you a full audit trail for compliance and debugging.

8. Practice task for Day 13

- Create three environments: Development, Staging, Production
- Add a required approval on Production (assign yourself as approver)
- Add a Branch Control check on Production: only allow main branch
- Create a 3-stage pipeline targeting all three environments
- Run the pipeline and approve the production stage

Today's cheat sheet	
Environment	Pipelines > Environments — tracks deployment history
deployment: job type	Required to target an environment and record history
Approval check	Human must approve before pipeline continues to that env
Branch control check	Only allow deployments from specific branches
runOnce strategy	Deploy to all targets at once — simplest strategy
Canary strategy	Deploy to small % first, verify, then full rollout

Day 14: Docker & Containers — Build and Deploy Container Images

1. What is Docker?

Docker packages your application and all its dependencies into a container — a lightweight, portable unit that runs identically on any machine. No more 'it works on my machine' problems.

2. Key Docker concepts

Image	A read-only blueprint for a container. Built from a Dockerfile.
Container	A running instance of an image. Isolated process.
Dockerfile	A text file with instructions to build an image.
Registry	A storage service for images (Docker Hub, Azure Container Registry).
Tag	A version label for an image (e.g. myapp:1.0, myapp:latest).
Push	Upload an image to a registry.
Pull	Download an image from a registry.

3. A simple Dockerfile

```
FROM node:20-alpine          # Base image
WORKDIR /app                 # Set working directory
COPY package*.json ./        # Copy package files
RUN npm install              # Install dependencies
COPY . .                     # Copy all source code
EXPOSE 3000                  # Open port 3000
CMD ["node", "index.js"]   # Start command
```

4. Essential Docker commands

```
docker build -t myapp:1.0 .          # Build image from Dockerfile
docker run -p 3000:3000 myapp:1.0    # Run container
docker ps                            # List running containers
docker images                        # List local images
docker push myregistry.io/myapp:1.0  # Push to registry
docker pull myregistry.io/myapp:1.0  # Pull from registry
docker stop <container-id>           # Stop a container
```

5. Azure Container Registry (ACR)

ACR is Microsoft's private Docker registry on Azure. You push images to ACR and pull them during deployment. It integrates natively with Azure Pipelines and Azure Kubernetes Service.

6. Docker in Azure Pipelines

```
trigger:
```

```
  - main

pool:
  vmImage: ubuntu-latest

variables:
  imageRepo: myapp
  tag: $(Build.BuildId)

stages:
  - stage: Build
    jobs:
      - job: DockerBuild
        steps:
          - task: Docker@2
            displayName: Build and push image
            inputs:
              command: buildAndPush
              repository: $(imageRepo)
              dockerfile: Dockerfile
              containerRegistry: MyACRServiceConnection
              tags: |
                $(tag)
                latest
```

7. Service connection to ACR

A Service Connection in Azure DevOps stores credentials to external services securely. Create a Docker Registry service connection pointing to your ACR so pipelines can push images without hard-coding credentials.

- Go to Project Settings > Service Connections > + New Service Connection
- Select Docker Registry > Azure Container Registry
- Select your subscription and ACR name
- Name the connection (e.g. MyACRServiceConnection)

8. Practice task for Day 14

- Install Docker Desktop on your local machine
- Write a Dockerfile for a simple HTML page served by nginx
- Build and run it locally: docker build -t mypage . && docker run -p 8080:80 mypage
- Browse to localhost:8080 and see your page
- Create an Azure Container Registry in the Azure Portal (free tier)

Today's cheat sheet	
Docker image	Packaged app + dependencies — portable
Docker container	Running instance of an image
Dockerfile	Recipe to build an image (FROM, RUN, COPY, CMD)
ACR	Azure Container Registry — private image storage

Docker@2 task	Build and push images in Azure Pipelines
Service connection	Secure credentials to external services (ACR, AWS, etc.)

Day 15: AKS & Kubernetes — Deploy to Azure Kubernetes Service

1. What is Kubernetes?

Kubernetes (K8s) is a system that automatically manages containers across many machines. It handles: running your containers, restarting them if they crash, scaling them up when traffic increases, and distributing traffic between them.

2. Key Kubernetes concepts

Cluster	A group of machines (nodes) managed by Kubernetes
Node	A single VM in the cluster that runs containers
Pod	The smallest unit — wraps one or more containers
Deployment	Declares how many pod replicas to run and which image
Service	A stable network endpoint that routes traffic to pods
Namespace	Virtual cluster inside K8s — used to separate environments
kubectl	Command-line tool to control Kubernetes clusters
YAML manifests	Files that describe desired state of K8s resources

3. What is AKS?

Azure Kubernetes Service (AKS) is Microsoft's managed Kubernetes offering. Azure handles the control plane (master nodes, API server, etcd) for free — you only pay for the worker VMs you run your workloads on.

4. Kubernetes deployment manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myacr.azurecr.io/myapp:$(tag)
          ports:
            - containerPort: 3000
```

```
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
```

5. Deploying to AKS in Azure Pipelines

```
- task: KubernetesManifest@0
  displayName: Deploy to AKS
  inputs:
    action: deploy
    kubernetesServiceConnection: MyAKSConnection
    namespace: production
    manifests: |
      manifests/deployment.yml
      manifests/service.yml
    containers: myacr.azurecr.io/myapp:$(Build.BuildId)
```

6. Common kubectl commands

```
kubectl get pods                      # List all pods
kubectl get deployments               # List deployments
kubectl get services                  # List services (see external IP)
kubectl logs <pod-name>              # View pod logs
kubectl describe pod <pod-name>      # Detailed pod info
kubectl rollout restart deployment/myapp  # Restart deployment
kubectl scale deployment myapp --replicas=5  # Scale to 5 pods
```

7. Practice task for Day 15

- Create a free AKS cluster in Azure Portal (1 node, cheapest VM size)
- Install kubectl locally and connect: az aks get-credentials --name MyCluster --resource-group MyRG
- Run: kubectl get nodes — see your cluster node
- Deploy the nginx image: kubectl create deployment nginx --image=nginx
- Expose it: kubectl expose deployment nginx --port=80 --type=LoadBalancer

Today's cheat sheet	
AKS	Azure Kubernetes Service — managed K8s control plane
Pod	Smallest K8s unit — wraps container(s)
Deployment	Manages desired # of pod replicas
Service	Stable IP/DNS that routes to pods
kubectl get pods	List all running pods
KubernetesManifest@0	Azure Pipelines task to deploy K8s manifests

Day 16: Infrastructure as Code — Terraform, ARM & Bicep with Azure Pipelines

1. What is Infrastructure as Code (IaC)?

IaC means defining your cloud infrastructure (virtual machines, databases, networks, storage) in code files, just like application code. Benefits: version-controlled, repeatable, consistent across environments, no manual clicks in the portal.

2. IaC options for Azure

Terraform (HashiCorp)	Open-source, multi-cloud (Azure, AWS, GCP). Most popular. Uses HCL language.
ARM Templates	Azure's native JSON-based IaC. Verbose but powerful.
Bicep	Microsoft's modern DSL that compiles to ARM. Much simpler than JSON ARM templates.
Pulumi	IaC using real programming languages (TypeScript, Python, Go).

3. Terraform basics

main.tf — Create an Azure Resource Group and Storage Account:

```
terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~>3.0"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "rg" {
  name     = "MyResourceGroup"
  location = "East US"
}

resource "azurerm_storage_account" "storage" {
  name                     = "mystorageacct2024"
  resource_group_name      = azurerm_resource_group.rg.name
  location                 = azurerm_resource_group.rg.location
  account_tier             = "Standard"
  account_replication_type = "LRS"
}
```

4. Terraform workflow

terraform init	Download providers and initialize the backend
terraform plan	Preview what changes will be made — shows + add, ~ modify, - destroy
terraform apply	Apply the changes — creates/updates/destroys resources
terraform destroy	Delete all resources managed by this configuration
terraform state	Inspect current known state of your infrastructure

5. Terraform in Azure Pipelines

```
steps:
  - task: TerraformInstaller@0
    inputs:
      terraformVersion: latest

  - task: TerraformTaskV2@2
    displayName: Terraform Init
    inputs:
      provider: azurerm
      command: init
      backendServiceArm: MyAzureServiceConnection
      backendAzureRmResourceGroupName: TerraformState-RG
      backendAzureRmStorageAccountName: tfstateaccount
      backendAzureRmContainerName: tfstate
      backendAzureRmKey: prod.terraform.tfstate

  - task: TerraformTaskV2@2
    displayName: Terraform Plan
    inputs:
      provider: azurerm
      command: plan
      environmentServiceNameAzureRM: MyAzureServiceConnection

  - task: TerraformTaskV2@2
    displayName: Terraform Apply
    inputs:
      provider: azurerm
      command: apply
      environmentServiceNameAzureRM: MyAzureServiceConnection
```

6. Remote state in Azure Blob Storage

Terraform keeps a state file that tracks what infrastructure it created. Store this file in Azure Blob Storage so it is shared across team members and pipelines — never commit it to Git.

7. Practice task for Day 16

- Install Terraform locally from terraform.io
- Write a main.tf that creates an Azure Resource Group
- Run terraform init, terraform plan, terraform apply
- Verify the resource group appears in the Azure Portal

- Run terraform destroy to clean up

Today's cheat sheet	
IaC	Infrastructure defined in code files, not manual portal clicks
Terraform	Most popular multi-cloud IaC tool (HCL language)
terraform plan	Preview changes before applying
terraform apply	Create/update real Azure resources
State file	Tracks what Terraform created — store in Azure Blob
Bicep	Microsoft's simpler alternative to ARM JSON templates

Day 17: Security & Permissions — Branch Policies, Service Connections & RBAC

1. Security in Azure DevOps — overview

Security in Azure DevOps spans four areas: who can access (users and groups), what they can do (permissions), how code is protected (branch policies), and how pipelines connect to external services (service connections).

2. Permission levels

Organization level	Controls who can create projects, manage billing, manage users
Project level	Controls access to Boards, Repos, Pipelines within a project
Object level	Controls access to specific repos, pipelines, environments, feeds
Pipeline level	Which pipelines can access which environments or service connections

3. Built-in security groups

Project Administrators	Full control of the project — add members, manage settings
Build Administrators	Can manage and run all pipelines
Contributors	Can push code, create PRs, run pipelines
Readers	Read-only access — can view everything but change nothing
Project Collection Administrators	Organization-level full control

4. Branch policies

Branch policies protect important branches from direct pushes and enforce code quality standards before merging.

- Go to Project Settings > Repositories > Select repo > Policies > Branch Policies
- Select the branch to protect (usually main or develop)
- Add policies:

Require minimum reviewers	Set minimum 1 or 2 approvers on every PR
Check for linked work items	PRs must reference a User Story or Bug
Check for comment resolution	All PR comments must be resolved before merge

Limit merge types	Allow only squash merge or rebase — no merge commits
Build validation	A specific pipeline must pass before PR can be merged

5. Service connections

A service connection securely stores credentials for external services (Azure subscriptions, Docker registries, GitHub, AWS, SonarCloud, etc.) so pipelines can access them without embedding secrets in YAML.

- Go to Project Settings > Service Connections > + New Service Connection
- Common types: Azure Resource Manager, Docker Registry, GitHub, SSH, Kubernetes
- Grant pipelines permission to use the service connection
- Reference in YAML: azureSubscription: 'MyServiceConnection'

6. Service connection security

Grant access to all pipelines	Any pipeline in the project can use this connection — convenient but less secure
Restrict to specific pipelines	Only named pipelines can use the connection — more secure for production

7. Personal Access Tokens (PAT)

A PAT is a password substitute for authenticating scripts, tools, or agents to Azure DevOps. Always set an expiry date. Use the minimum scope needed. Rotate periodically.

- Go to User Settings (top right) > Personal Access Tokens > + New Token
- Set expiry and select only the scopes needed
- Copy and store securely — Azure DevOps will not show it again

8. Practice task for Day 17

- Enable branch policies on your main branch: require 1 reviewer + build validation
- Try to push directly to main — it should be rejected
- Create a service connection to your Azure subscription
- Create a PAT with Code: Read scope and use it to clone via HTTPS

Today's cheat sheet	
Branch policy	Rules that protect a branch — enforced on every PR
Build validation	CI pipeline must pass before PR can be merged
Service connection	Secure credentials for external services
PAT	Personal Access Token — password substitute for tools

Contributors group	Default group for developers — push + PR access
Project Settings > Service Connections	Where all service connections are managed

Day 18: Monitoring & Reporting — Dashboards, Analytics & Azure Monitor

1. Monitoring in DevOps

Monitoring closes the feedback loop. After deploying, you monitor the running application to detect errors, track performance, understand user behavior, and catch problems before users report them.

2. Azure DevOps Dashboards

Dashboards are customizable pages in Azure DevOps where you add widgets to visualize team data — build results, work item counts, sprint burndown, deployment frequency, and more.

- Go to your project and click Dashboards in the top menu
- Click + Add Widget to browse available widgets
- Common widgets: Sprint Burndown, Build History, Deployment Status, Lead Time, Velocity
- Create team-specific dashboards (e.g. Dev Team Dashboard, QA Dashboard)

3. Pipeline analytics

Azure Pipelines has built-in analytics. Go to Pipelines > select a pipeline > Analytics tab. You can see: pass rate trend, duration trend, and failure analysis showing which stages fail most.

4. Azure Monitor

Azure Monitor is the central monitoring platform for Azure services. It collects metrics (CPU, memory, requests) and logs from your applications and infrastructure.

Metrics	Numeric values over time (e.g. CPU %, request count, response time)
Logs	Detailed event records (e.g. errors, warnings, traces)
Alerts	Notify you when a metric crosses a threshold (e.g. CPU > 80%)
Dashboards	Visual charts and graphs of metrics and logs
Workbooks	Interactive reports combining metrics, logs, and text

5. Application Insights

Application Insights is Azure Monitor's application performance monitoring (APM) component. Add its SDK to your app and it automatically tracks: request rates, failure rates, response times, exceptions, and user behavior — with no manual instrumentation needed for basics.

- Create an Application Insights resource in Azure Portal
- Get the Instrumentation Key or Connection String
- Add the SDK to your app and set the connection string

- Deploy and start seeing live telemetry in minutes

6. Key DORA metrics (DevOps Research & Assessment)

Deployment Frequency	How often you deploy to production (Daily/Weekly/Monthly)
Lead Time for Changes	Time from code commit to running in production
Change Failure Rate	% of deployments that cause an incident or rollback
Mean Time to Restore (MTTR)	How fast you recover from a production failure

NOTE: High performing teams deploy multiple times per day with < 15 min lead time and < 15% change failure rate.

7. Alerts and notifications

- In Azure Monitor, go to Alerts > + Create > Alert rule
- Select your resource (e.g. App Service, AKS)
- Define condition: e.g. HTTP 5xx errors > 10 per minute
- Define action group: send email, Teams notification, or call webhook
- Set alert severity: Critical, Error, Warning, Informational

8. Practice task for Day 18

- Create a dashboard in your Azure DevOps project with 4 widgets
- Add: Build History, Sprint Burndown, Work Item Count, Deployment Status
- Go to a pipeline's Analytics tab and review the pass rate chart
- In the Azure Portal, create an Application Insights resource and explore it

Today's cheat sheet	
Azure DevOps Dashboard	Custom widget-based views of team data
Pipeline Analytics	Pipelines > select pipeline > Analytics tab
Azure Monitor	Central platform for metrics and logs across Azure
Application Insights	APM — tracks requests, errors, response times in your app
DORA metrics	Industry standard measures of DevOps performance
Alert	Notification triggered when a metric crosses a threshold

Day 19: End-to-End Project — Full CI/CD Pipeline for a Real App

Goal of Day 19

Today you build a complete, working CI/CD pipeline from scratch. We will use a simple Node.js web app, containerize it, push to Azure Container Registry, and deploy to an Azure App Service — all automated through Azure Pipelines.

Architecture overview

Source code	Azure Repos (Git)
CI	Azure Pipelines — build Docker image on every push to main
Image storage	Azure Container Registry (ACR)
CD	Azure Pipelines — deploy image to Azure App Service
Approval gate	Required approval before deploying to Production
Secrets	Stored in Azure Key Vault, linked via Variable Group

Step 1: Set up the app code

Create these files in your Azure Repo:

```
app.js:
  const http = require('http');
  const server = http.createServer((req, res) => {
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.end('<h1>Hello from Azure DevOps Pipeline!</h1>');
  });
  server.listen(3000);

package.json:
  { "name": "myapp", "version": "1.0.0", "main": "app.js" }

Dockerfile:
  FROM node:20-alpine
  WORKDIR /app
  COPY . .
  EXPOSE 3000
  CMD ["node", "app.js"]
```

Step 2: Create Azure resources

- Create a Resource Group: MyApp-RG
- Create an Azure Container Registry: myappackregistry
- Create an Azure App Service Plan (Linux, B1 tier)
- Create an Azure App Service (Container type) targeting the plan
- Create a service connection in Azure DevOps to your Azure subscription
- Create a service connection to your ACR

Step 3: The complete pipeline YAML

```yaml
trigger:
  - main

variables:
  acrName: myappackregistry
  imageName: myapp
  tag: $(Build.BuildId)

pool:
  vmImage: ubuntu-latest

stages:
  - stage: Build
    displayName: Build and Push Image
    jobs:
      - job: BuildAndPush
        steps:
          - task: Docker@2
            displayName: Build and push to ACR
            inputs:
              command: buildAndPush
              repository: $(imageName)
              dockerfile: Dockerfile
              containerRegistry: ACRServiceConnection
              tags: |
                $(tag)
                latest

  - stage: DeployStaging
    dependsOn: Build
    displayName: Deploy to Staging
    jobs:
      - deployment: DeployStaging
        environment: Staging
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebAppContainer@1
                  displayName: Deploy container to Staging
                  inputs:
                    azureSubscription: AzureServiceConnection
                    appName: my-staging-app
                    containers: $(acrName).azurecr.io/$(imageName):$(tag)

  - stage: DeployProduction
    dependsOn: DeployStaging
    displayName: Deploy to Production
    jobs:
      - deployment: DeployProduction
        environment: Production
        strategy:
          runOnce:
            deploy:
              steps:
                - task: AzureWebAppContainer@1
                  displayName: Deploy container to Production
```

```
    inputs:
      azureSubscription: AzureServiceConnection
      appName: my-production-app
      containers: $(acrName).azurecr.io/$(imageName):$(tag)
```

Step 4: Wire up approvals

- Go to Environments > Production > Approvals and Checks
- Add Approval: assign yourself
- Add Branch Control: allow only main branch

Step 5: Run and verify

- Commit all files and push to main
- Watch the Build stage — image should be built and pushed to ACR
- Approve the Production deployment
- Open your App Service URL — see Hello from Azure DevOps Pipeline!

Today's cheat sheet	
Full pipeline flow	Code push > CI Build > Docker Image > ACR > Deploy > Approve > Production
AzureWebAppContainer@1	Deploy a Docker container to Azure App Service
Docker@2 buildAndPush	Build image and push to registry in one step
Environment approval	Required human gate before production deploy
tag: $(Build.BuildId)	Unique version tag for every build's image
dependsOn:	Ensure stages run in correct order

Day 20: AZ-400 Exam Prep — Key Topics, Practice Questions & Tips

About the AZ-400 exam

The Microsoft AZ-400: Designing and Implementing Microsoft DevOps Solutions exam validates your ability to combine people, process, and technologies to continuously deliver valuable products and services. It is the certification for Azure DevOps Engineers.

Exam format	40–60 questions, multiple choice, case studies, drag-and-drop
Duration	120 minutes
Passing score	700 out of 1000
Prerequisites	AZ-104 (Azure Admin) or AZ-204 (Azure Developer) recommended
Validity	1 year — must renew with a free online assessment
Cost	USD 165 (varies by country)

Exam domain breakdown

Configure processes and communications	~10%
Design and implement source control	~15%
Design and implement build and release pipelines	~40%
Develop a security and compliance plan	~10%
Implement an instrumentation strategy	~10%
Design and implement infrastructure as code	~15%

Critical topics to master

- YAML pipeline structure: triggers, stages, jobs, steps, conditions, dependsOn
- Variable groups, secret variables, and Azure Key Vault integration
- Branch policies: minimum reviewers, build validation, comment resolution
- Service connections: types, security, scope (all pipelines vs specific)
- Deployment strategies: runOnce, rolling, canary, blue-green
- Environment approvals and checks
- Docker: Dockerfile, build, push, ACR, task Docker@2
- Kubernetes: AKS, kubectl, KubernetesManifest@0, deployment manifests
- Terraform: init, plan, apply, state backend in Azure Storage

- Application Insights: SDK integration, Live Metrics, alerts
- DORA metrics: deployment frequency, lead time, MTTR, change failure rate
- Git strategies: branching models, merge types, PR workflow

Sample practice questions

Q1: You need to ensure the main branch can only receive code through PRs reviewed by at least 2 people. What do you configure?
Answer: Branch policies on the main branch with Require minimum reviewers set to 2.

Q2: A pipeline fails because it cannot authenticate to Azure Container Registry. What is the most secure fix?
Answer: Create a Docker Registry service connection in Project Settings > Service Connections pointing to the ACR, then reference it in the pipeline YAML.

Q3: You want to deploy to production only if staging succeeds AND a human approves. What do you configure?
Answer: Use dependsOn: DeployStaging with condition: succeeded() on the production stage, and add an Approval check on the Production environment.

Q4: Your Terraform state is getting corrupted when two engineers run terraform apply simultaneously. What should you do?
Answer: Configure the backend to use Azure Blob Storage with state locking enabled (Azure Blob supports native state locking with Terraform).

Q5: You want canary deployment where 10% of users get the new version first. What strategy do you use in Azure Pipelines?
Answer: Use the canary deployment strategy in your deployment job with the incrementPercentage set to 10.

Study resources

Microsoft Learn	learn.microsoft.com — free AZ-400 learning path
Azure DevOps Labs	azuredevopslabs.com — hands-on labs (free)
A Cloud Guru / Pluralsight	Paid video courses for AZ-400
MeasureUp / Whizlabs	Practice exam questions — very similar to real exam
Azure DevOps Docs	docs.microsoft.com/azure/devops — official reference

Exam day tips

- Read every question carefully — Microsoft often tests subtle distinctions
- Eliminate obviously wrong answers first — usually 2 out of 4 can be ruled out
- For architecture questions: security and scalability usually win over convenience
- Know the difference between Classic and YAML pipelines — exam tests both
- Branch policies questions are very common — know every policy type
- Service connection questions: always prefer service principal over personal credentials

- Time management: flag difficult questions and return — do not get stuck

Today's cheat sheet	
Passing score	700 / 1000
Exam duration	120 minutes
Heaviest domain	Build & release pipelines (~40%)
Key YAML keywords	trigger, stages, jobs, steps, dependsOn, condition
Best free resource	learn.microsoft.com — official Microsoft Learn
Practice labs	azuredevopslabs.com — completely free

You made it!

20 days of Azure DevOps — from zero to job-ready.

Keep practicing. Build real projects. Get certified. You have got this.